

ООП в Python

заметки по видео-урокам

Мурашев Илья

October 1, 2026

1 Введение

1.1 Объекты(данные и поведение)

- `type(obj)` – Выводит тип объекта. Каждый объект лежит в каком-либо классе.

```
1 a = 3
2 print(type(a))
3 << < class 'int' >
```

- `isinstance(obj, type)` – проверяет соответствие объекта `obj` переданному классу `type`, возвращает `bool`.

```
1 a = 3
2 print(isinstance(a, int))
3 << True
```

Заметка

В качестве типа(второго аргумента) можно передать параметр `object`, а в качестве `obj`(первого аргумента) любой тип. Тогда функция будет возвращать `True` на любом типе, что доказывает, что любой тип в Python является объектом.

1.2 Классы

Определение

Класс – шаблон, при помощи которого можно создавать объекты. В классе определяются и описываются объекты и их поведение.

```
1 class Car:
2     pass
```

Listing 1: пример создания пустого класса

```
1 class Car:
2     pass
3 a = Car()
4 print(type(a))
5 << __main__.Car
```

Listing 2: Пример создания экземпляра класса Car

В классе можно определить поля(по умолчанию они публичные):

```
1 class Car:
2     model = "BMW"
3     engine = 1.6
```

1.3 Атрибуты класса

```
1 class Person:
2     name = 'Ivan'
3     age = 30
```

Определение

Поля `name` и `age` называются атрибутами класса.

- `ClassName.__dict__` – позволяет выводить всю информацию(в том числе об атрибутах) о классе.

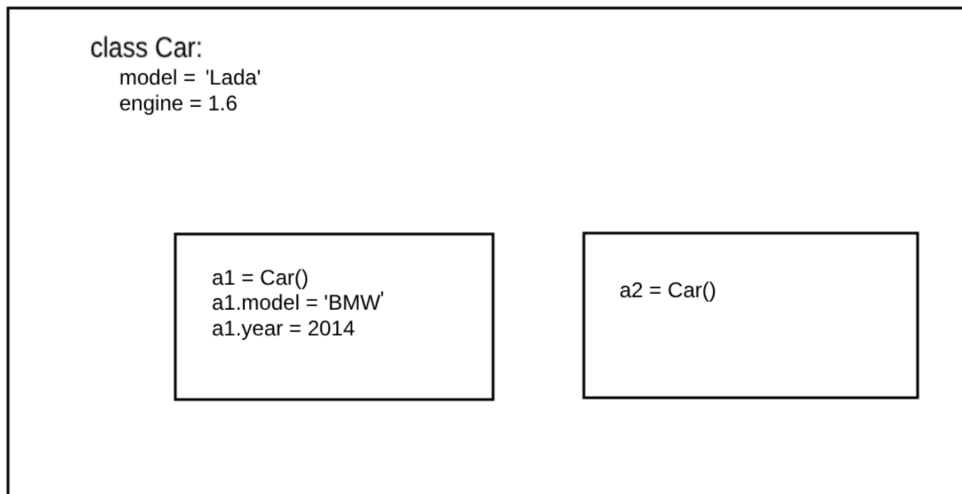
```
1 class Car:
2     model = "LADA"
3     engine = 1.6
4
5 print(Car.__dict__)
6 << {'__module__': '__main__', '__firstlineno__': 1, 'model': 'LADA',
    'engine': 1.6, '__static_attributes__': (), '__dict__': <attribute
    '__dict__' of 'Car' objects>, '__weakref__': <attribute
    '__weakref__' of 'Car' objects>, '__doc__': None}
```

- `getattr(Car, "model", 200)` – (название класса, название поля в кавычках, код ошибки) – безопасное обращение к полю: возвращает его значение или значение ошибки если поля нет. Можно прокидывать как класс, так и экземпляр класса.
- `setattr(ClassName, 'AttributeName', value)` – позволяет добавлять поле в созданный ранее класс.
- `ClassName.NewAttribute = value` – такой синтаксис создания атрибута класса тоже работает. похоже на объявление новой переменной.
- `delattr(ClassName, 'AttributeName')` – позволяет удалять поле внутри в созданного ранее класса.
- `del ClassName.AttributeName` – аналогичный синтаксис удаления поля класса
- `hasattr(ClassName, AttributeName)` – проверяет, есть ли у переданного класса заданный атрибут, возвращает `bool`.

Заметка

`setattr`, `delattr` при вызове объекта самого класса изменяют значения у всех экземпляров класса. Их можно вызывать для самих экземпляров, тогда изменения будут локальны.

1.4 Атрибуты экземпляра класса



В примере есть класс `Car` и его два экземпляра `a1` и `a2`. `a1` наследует все поля родителя. Может их изменять и создавать новые локально в своей области видимости. `a2` пуст и все поля, которые есть у него – это копии полей родителя.

1.5 Функция как экземпляр класса

Внутри класса(не экземпляра класса) можно объявлять как поля, так и методы(функции).

```

1  class Car:
2      model = 'BMW'
3      engine = 1.6
4
5      def drive():
6          print("Let's go!")
7
8  Car.drive() # <-- reference to class name, not to variable
9  << Let's go!
  
```

Заметка

При таком написании вывод функции можно увидеть только если обратиться именно к самому классу `Car` и к **функции** внутри него. Если создать экземпляр класса `b = Car()` и вызвать уже **метод** `b.drive()` компилятор вернет ошибку: `TypeError: Car.drive() takes 0 positional arguments but 1 was given`. Данная реализация функции не принята!

Для лечения такой вещи можно воспользоваться декоратором `@staticmethod`:

```

1  class Car:
2      model = 'BMW'
3      engine = 1.6
4
5      @staticmethod
6      def drive():
7          print("Lets go!")
8
9  Car.drive() # <-- reference to class name
10 << Lets go!
11 b = Car()
  
```

```

12     b.drive()      # <-- reference to class variable name
13     << Lets go!

```

1.6 Методы класса. Аргумент self.

Заметим интересную деталь:

```

1     class Cat:
2     def hello():
3         print('Hello, everyone!')
4
5     Bob = Cat()
6     print(Cat.hello)
7     print(Bob.hello)
8
9     << <function Cat.hello at 0x70e2a839f270>
10    << <bound method Cat.hello of <__main__.Cat object at 0x70e2a8390d70>>

```

Главные отличия понятия **функции** класса и **метода** класса:

- Метод – та же функция, только объявленная внутри класса.
- Метод привязан к конкретному объекту(объект Bob в примере). Функция привязана целиком к классу и вызывается от него(класс Cat).
- При вызове метода, тот объект, который с ним связан, автоматически прокидывается внутрь метода в качестве аргумента.

```

1     class Cat:
2     def hello(*args):
3         # = f(...), get random
4         # amount of arguments
5         print('Hello, everyone!', args) # output given arguments
6
7     Bob = Cat()
8     print(Cat.hello())
9     print(Bob.hello())
10
11    << Hello, everyone! ()
12    << Hello, everyone! (<__main__.Cat object at 0x7c5144730ec0>,)

```

Вывелся адрес хранения объекта Bob в памяти.

Заметка

При вызове метода обязательно(согласно синтаксису) внутрь передается 1 аргумент – сам объект класса. (*this прокидывается всегда и именуется первым аргументом). Его обычно называют `self`, однако это лишь условно и название этого параметра может быть произвольно.

```

1 class Cat:
2     breed = 'pers'
3     age = 5
4     name = 'default'
5     def SetCat(self, age, name):
6         self.age = age
7         self.name = name
8     def Hello(self):
9         print(f"Hello, {self.name}!")
10
11 Bob = Cat()
12 Bob.SetCat(6, 'Bob')
13 Bob.Hello()
14
15 << Hello, Bob!

```

Создали класс `Cat` с атрибутами `breed`, `age`, `name`. После этого создали метод-конструктор, который присвоил указанным полям переданные значения. Обращается он к экземпляру класса – к `self` (аналог `*this`).

Заметка

метод `SetCat` нельзя вызывать без аргументов `age`, `name`. Иначе это приведет к ошибке. Можно использовать = default значения параметров:
`def SetCat(self, age = 5, name = 'Stiv') :`

1.7 Инициализация объекта. Метод `init`.**Определение**

Метод `__init__` – специальный метод классов, предназначенный для инициализации полей объекта(= конструктор). Вызывается сразу после создания экземпляра класса.

```

1 class Cat:
2     breed = 'pers'
3     age = 5
4     name = 'default'
5     def __init__(self, breed, age, name):    #constructor
6         self.breed = breed
7         self.age = age
8         self.name = name
9     def Hello(self):
10        print(f"Hello, {self.name} {self.breed} {self.age}!")
11 Bob = Cat('Russian', 6, 'Ivan')           #construct specific object Bob
12 Bob.Hello()                               #call method
13 << Hello, Ivan Russian 6!

```

Теперь метод `SetCat` полностью убрался в конструктор `__init__`

Заметка

Обычно, аналогично с++, имена переменных в конструкторе называют аналогично соответствующим инициализации параметрам.

В качестве примера создадим класс точки. Добавим конструктор и методы: напечатать точку, вывести расстояние до другой точки, и переместиться в другую точку.

```

1 class Point:
2     x = 0
3     y = 0
4
5     def move_to(self, new_x, new_y):
6         self.x = new_x
7         self.y = new_y
8
9     def __init__(self, coord_x = 0, coord_y = 0): #(0,0) = default
10        self.move_to(coord_x, coord_y)
11
12    def go_home(self):
13        self.move_to(0,0)
14
15    def print_coord(self):
16        print(f"Точка скоординатами ({self.x},{self.y})")
17
18    def dist(self, point):
19        if(not isinstance(point, Point)):
20            raise ValueError("Аргумент должен принадлежать классу Point")
21        return ((self.x - point.x)**2 + (self.y - point.y)**2)**0.5

```

1.8 Моносостояния

В Python нет ссылок как в C++. Однако, например, для словарей по умолчанию операция `=` означает привязка обоих имен к одному объекту в памяти (аналог ссылки), а не копирование одного в другой. В силу этого можно реализовать следующую структуру: у класса `Car` общий словарь `__shared_attr` с полями `'brand'` и `'volume'`. При создании в `__init__` каждый экземпляр класса состоит из буквально этого же словаря. Поэтому изменения полей в одном влекут изменения в другом.

```

1 class Car:
2     __shared_attr = {
3         'brand': 'Lada',
4         'volume': 3
5     }
6
7     def __init__(self):
8         self.__dict__ = Car.__shared_attr
9
10    def info(self):
11        print(f"Машина марки {self.__shared_attr['brand']} и размера {self.__shared_attr['volume']}")
12
13    b = Car()
14    b.__shared_attr['volume'] = 5
15    c = Car()
16    c.info()
17
18 << Машина | марки | Lada и | размера | | 5

```

1.9 Модификаторы доступа

В классах в Python существуют модификаторы доступа полей и методов:

- `public` - поля. По умолчанию все поля и методы в статусе `public`. Доступ к ним открыт вне класса.
- `protected` - поля. Просто договоренность между программистами, что отмеченная переменная или метод используются только для внутреннего устройства класса. Доступ вне класса такой же как у `public`. Для создания `protected` переменной или метода нужно поставить `_` перед названием.
- `private` - поля. поля и методы, защищенные от внешнего доступа. Для создания `private` переменной или метода нужно поставить `__` перед названием.

```
1 class Car:
2     def __init__(self, engine, wheel, price):
3         self.engine = engine
4         self._wheel = wheel
5         self.__price = price
6
7 b = Car(1.6, 35, 1000)
8 print(b.engine)
9 print(b._wheel)
10 print(b.__price)
11 << 1.6
12 << 35
13 << Traceback (most recent call last):
14   File "/home/daydreamer/snap/pyp_prog/test_2.py", line 10, in <module>
15     print(b.__height)
```

Определение

Инкапсуляция – важное свойство ООП, позволяющее скрывать внутреннее устройство классов и методов извне класса(существование `private` и `public` полей).

1.10 Property. Getter and setter

Для реализации идеи инкапсуляции так же используется специальный метод `property`. С его помощью можно использовать логику доступа, создания и удаления private переменных извне класса. Обычно он может включать в себя какую-то нетривиальную логику по созданию, выводу и удалению полей класса.

```

1 class BankAccount:
2     def __init__(self, name, balance):
3         self.__name = name          #private - var
4         self.__balance = balance    #private - var
5
6     def get_balance(self):
7         return self.__balance
8
9     def set_balance(self, new_balance):
10        if not isinstance(new_balance, (int, float)):
11            raise ValueError('Баланс должен быть числом')
12        self.__balance = new_balance
13    def del_balance(self):
14        del self.__balance
15
16        balance = property(fget=get_balance, fset=set_balance, fdel=del_balance)
17
18 q = BankAccount("Ivan", 200)
19 print(q.balance)                # access to get private balance
20 q.balance = 300                  # access to set private balance
21 print(q.balance)
22 del q.balance                    # access to del private balance
23 print(q.__dict__)
24 q.balance = 300
25 print(q.__dict__)
26 << 200
27 << 300
28 << {'_BankAccount__name': 'Ivan'}
29 << {'_BankAccount__name': 'Ivan', '_BankAccount__balance': 300}

```

Пример использования метода `property`

Тот же функционал можно получить немного по-другому:

```

1     balance = property(fget=get_balance, fset=set_balance, fdel=del_balance)
2
3     #----- balance == my_balance
4
5     my_balance = property()
6     my_balance = my_balance.getter(get_balance)
7     my_balance = my_balance.setter(set_balance)
8     my_balance = my_balance.deleter(del_balance)

```

1.11 Декораторы

1.12 Вложенные функции

В Python существует возможность создавать вложенные функции(в другие функции). При этом работают обычные правила области видимости. От более глобального к более частному. При поиске соответствия переменной программа рекурсивно поднимается вверх в надежде найти указанную переменную.

```

1
2 g = 'GRAY'
3 def colors():
4     # local function scope
5     g = 'GREEN'
6     y = 'YELLOW'
7     def print_red():
8         # = global but for f(f(x))
9         nonlocal y
10        r = 'RED'
11        print(r, y, g)
12        y = 'was changed'
13
14    def print_blue():
15        # more local function scope
16        b = 'BLUE'
17        print(b, y, g)
18
19    print_red()
20    print_blue()
21
22 #global scope
23 colors()
24
25 << RED YELLOW GREEN
26 << BLUE was changed GREEN

```

Заметка

- `global` – обращение внутри функции к глобальной переменной.
- `nonlocal` – обращение к переменной функции выше.

```

1 y = 10
2 def main_func():
3     global y
4     a = 67
5     def f():
6         nonlocal a # a from main_func
7         global y   # global var
8         print(a, y)
9     f()
10 main_func()

```

1.13 Замыкания

Заметка

Если функция ничего не возвращает(в ней нет `return`), то по умолчанию она вернет значение `None`.

В Python есть возможность отдавать указатели на функцию. В основной функции можно создать вспомогательную и в качестве выходного параметра отдавать её же.

```
1 def main_func():
2     name = 'Ivan'
3     def inner_func():
4         print('Hello, my friend!', name)
5
6     return inner_func
7
8 d = main_func()
9 d()
10
11 << Hello, my friend! Ivan
```

Определение

Замыкание – синтаксис в Python, позволяющий использовать во внутренней функции переменные из внешней, которые не объявлены в области видимости вложенной функции.

В таком случае локальная переменная `main_func` не удалится сразу после её работы(как обычно происходит со всеми локальными переменными). Таким образом в `main_func` можно передавать параметр и инстанцировать `inner_func` каждый раз с новым параметром

Примеры использования:

```
1 def main_func(value):
2     name = value
3     def inner_func():
4         print('Hello, my friend!', name)
5
6     return inner_func
7
8 d = main_func('Ivan')
9 d()
10 b = main_func('Vasya')
11 b()
12
13 << Hello, my friend! Ivan
14 << Hello, my friend! Vasya
```

```
1 def main_func(value):
2     def inner_func(a):
3         return value + a
4
5     return inner_func
6
7 c = main_func(10)
8 print(c(5))
9 << 15
```

Мы инстанцировали `main_func(10)` и аргумент, переданный в `inner_func` складывается с аргументом `main`.

```
1 def main_func():
2     counter = 0
3     def inner_func():
4         nonlocal counter
5         counter += 1
6         return counter
7
8     return inner_func
9
10 a = main_func()
11 print(a())
12 a()
13 a()
14 print(a())
15 << 1
16 << 4
```

Пример использования замыкания. Инициализируется `timer()`, после каждого нового выхода `q` возвращается время с момента её инициализации.

```
1 from datetime import datetime
2
3 def timer():
4     start = datetime.now()
5     def inner():
6         return datetime.now() - start
7     return inner
8
9 q = timer()
10 print(q())
11 print(q())
```

Еще один пример использования. Например, у нас есть функция с некой логикой и мы хотим посчитать, сколько раз она вызывалась. Тогда можно использовать структуру замыканий. Для корректного вызова функции использовались аргументы `*args, **kwargs`.

```
1 from datetime import datetime
2
3 def add(a,b):
4     return a + b
5
6 def counter(func):
7     count = 0
8     def inner(*args, **kwargs):
9         nonlocal count
10        count += 1
11        print(f"function {func.__name__} was called {count} times")
12        return func(*args, **kwargs)
13    return inner
14 q = counter(add)
15 q(10,20)
16 q(20,45)
17
18 << function add was called 1 times
19 << function add was called 2 times
```

1.14 Декораторы

Определение

Декоратор – функция, которая принимает на вход функцию, добавляет ей определенный функционал или поведение и возвращает функцию.

Простейший пример использования Декоратора. Мы хотим добавить к выводу `say()` некоторые дополнительные строки. Пишем замыкание и сохраняем результат в нашу же `say`.

```
1
2 def say():
3     print("Hello , World!")
4
5 def decator(func):
6     def inner():
7         print("start decorator....")
8
9         func()
10
11        print("end decorator....")
12    return inner
13
14 say = decator(say) #<-- inner keeps here
15 say() #<-- add things to say
```

Заметка

Стоит придерживаться правила: если внутри замыкания(для использования декоратора) используется функция, её аргументы полезно пробрасывать, используя `*args, **kwargs`

Декораторы можно использовать, например, для разметки `HTML` текста. С его помощью можно накидывать заголовки на блоки текста.

```
1
2 def say(name, surname, age):
3     print("Hello , ", name, surname, age)
4
5 def header(func):
6
7     def inner(*args, **kwargs):
8         print("<h1>")
9
10        func(*args, **kwargs)
11
12        print("<h1>")
13    return inner
14
15 def table(func):
16
17     def inner(*args, **kwargs):
18         print("<table>")
19
20        func(*args, **kwargs)
21
22        print("<table>")
23    return inner
24
```

```

25 say = table(header(say)) #<-- inner keeps here
26 say("Ivan", "Ivanov", 37)
27
28 << <table>
29 << <h1>
30 << Hello, Ivan Ivanov 37
31 << <h1>
32 << <table>

```

В разработке принято, что декораторы не пишут отдельной строчкой а навешивают, используя "@" – просто синтаксический сахар.

```

1 @table
2 @header # say = table(header(say))
3 def say(name, surname, age):
4     print("Hello, ", name, surname, age)

```

Заметка

Существует проблема: при декорировании функции теряется информация о ней: имя функции и документация(если есть).

```

1 def table(func):
2
3     def inner(*args, **kwargs):
4         print("<table>")
5         func(*args, **kwargs)
6         print("<table>")
7
8     return inner
9 def say():
10     print("Hello!")
11
12 q = table(say)
13 print(q)
14 << <function table.<locals>.inner at 0x79775fbae6c0>

```

Заметка

- `func.__name__` – вызвать название функции.
- `func.__doc__` – напечатать документацию функции(написать документацию можно сразу после объявления, используя многострочный комментарий).

существует два способа решения данной проблемы:

1. Вручную:

```

1 def table(func):
2
3     def inner(*args, **kwargs):
4         print("<table>")
5         func(*args, **kwargs)
6         print("<table>")
7
8     inner.__name__ = func.__name__ #changes inner's name
9     inner.__doc__ = func.__doc__  #changes inner's doc
10    return inner

```

2. Декоратор wraps

```

1 from functools import wraps
2
3 def table(func):
4
5     @wraps(func)
6     def inner(*args, **kwargs):
7         print("<table>")
8         func(*args, **kwargs)
9         print("<table>")
10
11     return inner

```

1.15 Использование декораторов для сеттеров и геттеров в ООП

пример с классом `BankAccount` можно изменить, используя уже декоратор `property`. Задача – иметь доступ на доступ, создание и удаление `private` поля через наши ручки доступа – `getter`, `setter`, `deleter`. Для этого можно написать следующий код:

```

1 class BankAccount:
2     def __init__(self, name, balance):
3         self.__name = name          #private - var
4         self.__balance = balance    #private - var
5
6     @property
7     def my_balance(self):
8         print("get_balance: ")
9         return self.__balance
10
11     my_property_balance = my_balance
12
13     def my_balance(self, new_balance):
14         if not isinstance(new_balance, (int, float)):
15             raise ValueError('Баланс должен быть числом')
16         print("set_balance: ")
17         self.__balance = new_balance
18
19     my_balance = my_property_balance.setter(my_balance)

```

Функции доступа придется называть одинаково, тк отличаются лишь операции, применяемые к `my_balance`. В связи с этим возникает конфликт имен и приходится выворачиваться, используя дополнительную переменную.

К счастью, проблема эта решена при помощи специальных декораторов, которые, по сути, делают это же, только скрытно:

```

1 class BankAccount:
2     def __init__(self, name, balance):
3         self.__name = name          #private - var
4         self.__balance = balance    #private - var
5
6     @property
7     def my_balance(self):
8         print("get_balance: ")
9         return self.__balance
10
11     @my_balance.setter
12     def my_balance(self, new_balance):
13         if not isinstance(new_balance, (int, float)):
14             raise ValueError('Баланс должен быть числом')

```

```
14     print("set_balance: ")
15     self.__balance = new_balance
16
17     @my_balance.deleter
18     def my_balance(self):
19         print("del_balance: ")
20         del self.__balance
21
22
23 q = BankAccount("Ivan", 200)
24 print(q.my_balance)           # access to get private balance
25 q.my_balance = 300           # access to set private balance
26 print(q.my_balance)
27 del q.my_balance             # access to del private balance
28 print(q.__dict__)
29 q.my_balance = 300
30 print(q.__dict__)
31 << get_balance:
32 << 200
33 << set_balance:
34 << get_balance:
35 << 300
36 << del_balance:
37 << { '_BankAccount__name': 'Ivan ' }
38 << set_balance:
39 << { '_BankAccount__name': 'Ivan ', '_BankAccount__balance': 300 }
```

1.16 @staticmethod и @classmethod

Пусть мы хотим реализовать в классе функцию, которая не зависит от объектов класса. Мы хотим уметь вызывать её как от самого класса, так и от экземпляров класса. Обычная реализация выдаст ошибку:

```

1 class Person:
2     def Say_self(self):
3         print(f"Hello_1")
4
5     def Say():
6         print("Hello_2")
7
8 Person.Say()
9 Person.Say_self()
10
11 q = Person()
12 q.Say()
13 q.Say_self()
14 <<      Person.Say_self()
15 <<      ~~~~~^
16 << TypeError: Person.Say_self() missing 1 required positional << argument: '
17 <<      self'
18 <<      q.Say()
19 <<      ~~~~~^
20 << TypeError: Person.Say() takes 0 positional arguments but 1 was given

```

Для обхода таких проблем можно использовать декоратор `@staticmethod`

```

1 class Person:
2     @staticmethod
3     def method_stat():          #no arguments
4         print("hello_stat!")
5
6     @classmethod
7     def method_cls(cls):       #cls = class
8         print(f"hello, {cls.__name__}")
9
10 Person.method_stat()
11 Person.method_cls()
12 q = Person()
13 q.method_stat()
14 q.method_cls()
15 << hello_stat!
16 << hello, Person
17 << hello_stat!
18 << hello, Person

```

Определение

Декоратор `@staticmethod` нужен для реализации внутри класса функции, которая может вызываться как и от класса, так и от экземпляра класса.

Определение

Декоратор `@classmethod` нужен для реализации внутри класса функции, которая производит обработку не над экземплярами, а над классом целиком с помощью вспомогательной переменной `cls`.